Evaluation Fundamentals

Specifying Behavior Through Testing

Bryan Bischof

4 Feb 2026

Today's Journey

1. **The Eval Mindset**
   - Analyze–Measure–Improve lifecycle
2. **Key Definitions**
   - Tasks, golden datasets, traces, failure modes, evaluators
3. **Running Example**
   - Support triage end-to-end
4. **Define "Good" & Build Data**
   - Rubrics, red lines, golden datasets
5. **Analyze: Traces to Failure Modes**
   - Open coding and axial coding
6. **Evaluators, Measure, Improve**
   - Deterministic evaluators, metrics, failure analysis
7. **Agreement & LLM-as-Judge**
   - Human calibration, model-based evaluation
8. **Pitfalls in Evals**
   - Goodharting, overfitting, metric laundering

Outline

1. The Eval Mindset
2. Key Definitions
3. Running Example: Support Triage
4. Step 1: Define "Good"
5. Step 2: Build a Golden Dataset
6. Analyze: From Traces to Failure Modes
7. Step 3: Evaluators
8. Step 4: Measure
9. Step 5: Failure Analysis

Evaluation Is a Method of AI Engineering

Evaluation is the way we **specify behavior**, **detect regressions**, and **decide what to improve next**.

The Eval Mindset

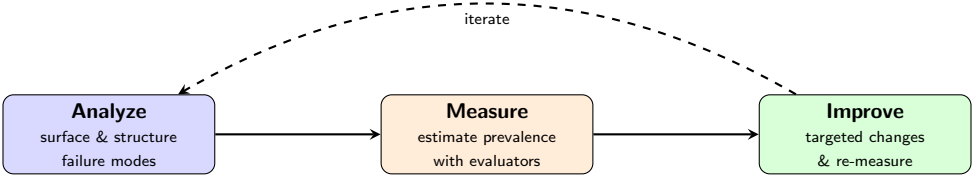
- **Experimenter mindset:** run experiments, inspect failures, then generalize
- **Data mindset:** trust data and examples more than vibes
- **Engineering mindset:** evals should be quick to run and easy to understand

Key Principle

If you can't measure it, you can't improve it. Evals turn "I think this works" into "I know where this fails."

Analyze–Measure–Improve Lifecycle

The evaluation lifecycle is **application-centric** and iterative:



1. **Analyze**: surface and structure failure modes (using traces + coding)
2. **Measure**: estimate prevalence of failure modes with automated evaluators
3. **Improve**: make targeted changes and re-measure (tight iteration loop)

This lecture walks through Analyze → Measure, then shows how Improve plugs back into the loop.

P^3 (Unit 1)

This entire unit lives in the **Proof** phase: rubric from Promise, golden dataset encodes failure model, evaluators test red lines.

Outline

1. The Eval Mindset
2. Key Definitions
3. Running Example: Support Triage
4. Step 1: Define "Good"
5. Step 2: Build a Golden Dataset
6. Analyze: From Traces to Failure Modes
7. Step 3: Evaluators
8. Step 4: Measure
9. Step 5: Failure Analysis

Task

Task: what the system must do for a user (input → output).

Example Tasks

Task	Input	Output
Support triage	{subject, body}	{category, priority}
Structured extraction	messy text	JSON {name, email, issue_type}
Summary + todos	meeting notes	{summary_bullets, todos}
Data questions	question	SQL query (or {query, explanation})

Why This Matters

A clear task definition is the prerequisite for everything else: golden datasets, evaluators, and failure analysis all flow from the task spec.

Golden Dataset

Golden dataset: a small, versioned test set encoding expected behavior.

Example Golden Datasets

- 50 support tickets covering typical + high-risk scenarios
- 30 bug reports for different features
- 40 meeting notes with expected todos + key summary points
- 25 data questions with correct data queries

Golden Dataset $\neq$ Benchmark

A **benchmark** is for broad comparability. A **golden dataset** encodes *your* expected behavior and failure modes.

Trace

Trace: the full record needed to evaluate behavior for one "case."

Example Traces

- One support ticket plus the model's triage output
- One doc/email plus extracted fields
- One meeting note plus summary + todos output
- One data question plus generated query and final answer

Key Insight

Traces are the raw material for everything downstream—you read them to find failures, code them into categories, and feed them into evaluators.

Failure Mode and Open Code

Failure mode: a recurring way the system fails.

Examples

- Triage: "misclassifies account takeover as `other`"
- Extraction: "missing required field (`email` is null/empty)"
- Summary/todos: "drops an action item that was clearly stated"
- Data questions: "wrong aggregation or missing filter in the query"

Open code: a quick, informal label you write while reading traces.

Examples

- Triage: "missed urgent signal"
- Extraction: "field missing / wrong type"
- Summary/todos: "lost key action item"
- Data questions: "query missing constraint"

Failure-mode taxonomy: a small, stable set of failure modes after merging open codes (axial coding).

Examples: "missed high-risk intent," "missing required fields," "omitted key action item," "incorrect query semantics"

Evaluator and Invariant

Evaluator: per-example check producing metrics and/or pass/fail.

Invariant: deterministic evaluator; cheap check that should *always* hold.

Examples

- Schema validator: JSON parses and required fields exist
- Exact match on a key field (e.g. `issue_type`, `priority`)
- Query checker: only `SELECT`, no destructive statements
- Latency/cost check: complete under X seconds/tokens

Examples

- Triage: output category and priority are in the allowed sets
- Extraction/todos: output must be valid JSON with required keys
- Summary: must preserve critical entities (names, dates, amounts)
- Data questions: query must be read-only (`SELECT` only)

Relationship

Every invariant is an evaluator, but not every evaluator is an invariant. Invariants are your "unit tests"—cheap, deterministic, and always expected to pass.

Rubric

Rubric: definition of "good" for subjective tasks (3–5 criteria).

Example Rubrics by Task

Task	Rubric Criteria
Triage	correctness, high-risk handling, clarity
Extraction	correctness, completeness, schema adherence
Summary/todos	faithful, complete, concise, structured
Data questions	correct intent, correct query, clear explanation

Why Rubrics Matter

Without a rubric, "good" is undefined. Rubrics align the team, calibrate human labelers, and provide criteria for LLM-as-judge evaluators.

Outline

1. The Eval Mindset
2. Key Definitions
3. Running Example: Support Triage
4. Step 1: Define "Good"
5. Step 2: Build a Golden Dataset
6. Analyze: From Traces to Failure Modes
7. Step 3: Evaluators
8. Step 4: Measure
9. Step 5: Failure Analysis

Running Example: Support Triage

Task:

- **Input**: support ticket {`subject, body`}
- **Output**: labels {`category, priority`}

Golden dataset: `examples/evals/data/golden_support_triage.jsonl`

Example row

```
{
  "example_id": "t05",
  "input": {
    "subject": "Possible account takeover",
    "body": "I see logins from a country I've never
             been to and my email was changed. URGENT."
  },
  "expected": {"category": "security", "priority": "high"}
}
```

Discussion

Why is this a high-risk case? What is the cost of misclassifying it?

Outline

1. The Eval Mindset
2. Key Definitions
3. Running Example: Support Triage
4. Step 1: Define "Good"
5. Step 2: Build a Golden Dataset
6. Analyze: From Traces to Failure Modes
7. Step 3: Evaluators
8. Step 4: Measure
9. Step 5: Failure Analysis

Step 1: Define "Good" (Rubric + Red Lines)

Prompt the room:

- If we ship this system, what are the top 3 user complaints?
- Which failures are *unacceptable* vs merely *annoying*?

Write down (as the spec):

1. **Rubric (3–5 criteria)**: what "good" means
2. **Red lines (3 items)**: unacceptable failures
3. **Hypotheses (5–10)**: where we expect the system to fail

Key Insight

Before you write a single line of eval code, you need agreement on what "good" and "unacceptable" mean. This is the specification step.

Example Rubric and Red Lines

Rubric for Triage

- **Correctness**: right category/priority
- **Safety**: never down-rank takeover / urgent security issues
- **Helpfulness**: outputs are actionable (clear category/priority)

Red Lines

- Mark account takeover as low priority
- Classify account takeover as "other"

Hypotheses (Where We Expect Failure)

- Ambiguous tickets may get wrong category
- Urgency signals buried in the body may be missed
- Security-related tickets may be classified as "account" or "other"

Outline

1. The Eval Mindset
2. Key Definitions
3. Running Example: Support Triage
4. Step 1: Define "Good"
5. Step 2: Build a Golden Dataset
6. Analyze: From Traces to Failure Modes
7. Step 3: Evaluators
8. Step 4: Measure
9. Step 5: Failure Analysis

Step 2: Build a Golden Dataset (Coverage + Diversity)

Coverage checklist (aim for diversity across scenarios):

1. **Typical cases** (most users)
2. **High-risk cases** (red lines)
3. **Ambiguous cases** (needs clarification)
4. **Messy cases** (real wording, long tail)

Golden dataset vs benchmark:

- **Benchmark**: broad comparability across systems
- **Golden dataset**: *your* expected behavior and failure modes

The golden dataset is a *living document*—you add cases as you discover new failure modes.

Micro-Activity (5–10 minutes)

- Pick one task and write 1–2 open codes for 5 examples
- Add 3 new examples that break the baseline

Outline

1. The Eval Mindset
2. Key Definitions
3. Running Example: Support Triage
4. Step 1: Define "Good"
5. Step 2: Build a Golden Dataset
6. Analyze: From Traces to Failure Modes
7. Step 3: Evaluators
8. Step 4: Measure
9. Step 5: Failure Analysis

Analyze: From Traces to Failure Modes

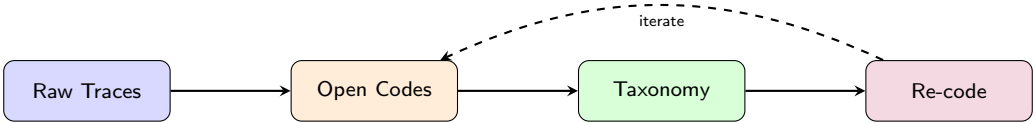
The goal is to turn raw traces into a small set of failure modes you can **measure**, **fix**, and **regression test**.

Unit 2

Hallucination is often one of the most common failure modes—factual fabrication and confident nonsense deserve their own taxonomy branch.

Steps:

1. Create a trace dataset (real, synthetic, or mixed)
2. Read traces and **open code** failures
3. **Axial code**: merge open codes into a failure-mode taxonomy
4. Re-code traces using the refined taxonomy (iterate until stable)



Mini Example: Coding in Practice

Start with 20 traces for one task (e.g. data questions):

Open code failures:

- "missing filter"
- "wrong join"
- "wrong time window"

These are quick, informal labels written *while reading traces*.

Axial code into taxonomy:

- "incorrect query semantics"
- "missing constraint"
- "wrong aggregation"

Merge and refine until the taxonomy is *stable* and covers observed failures.

The Process

Open coding → axial coding is borrowed from qualitative research. The goal is a **small, actionable** set of failure categories—not an exhaustive list.

Outline

1. The Eval Mindset
2. Key Definitions
3. Running Example: Support Triage
4. Step 1: Define "Good"
5. Step 2: Build a Golden Dataset
6. Analyze: From Traces to Failure Modes
7. Step 3: Evaluators
8. Step 4: Measure
9. Step 5: Failure Analysis

Step 3: Evaluators (Start Deterministic)

Deterministic evaluators are unit tests for AI behavior:

- Fast, cheap, stable
- Catch regressions immediately

What to evaluate depends on task type:

Structured Tasks

- JSON validity / schema checks
- Required fields present
- Exact match on key fields

Subjective Tasks

- "Must include" requirements (key points / todos)
- Entity preservation (names, dates, amounts)
- Length / format constraints

Start Here

Always start with deterministic evaluators. They are the cheapest, fastest way to catch obvious failures before investing in more expensive evaluation.

Worked Example: Triage Invariant

Invariant: output uses allowed labels.

Fail

```
{"category": "payments",
 "priority": "urgent"}
```

`"payments"` and `"urgent"` are not in the allowed label sets.

Pass

```
{"category": "billing",
 "priority": "high"}
```

Both values are valid members of the allowed sets.

Discussion:

- What does this invariant catch? → Output schema violations
- What does it miss? → Correct labels but wrong classification

Worked Example: Extraction Invariant

Invariant: valid JSON with required keys.

Fail

`"name: Alice, email: alice@x.com"`

Not valid JSON—cannot be parsed or validated.

Pass

```
{"name": "Alice",
 "email": "alice@x.com",
 "issue_type": "billing"}
```

Valid JSON with all required keys present.

Takeaway

Schema validation is the simplest invariant: does the output parse? Are all required fields present? This catches a large class of failures before you even look at correctness.

Worked Examples: Summary and Data Questions

Summary/todos invariant: preserve critical entities.

Fail

"Meeting is on **Friday**."
(Original said Thursday)

Pass

Includes "Thursday" and the right names/dates.

Data questions invariant: query is read-only.

Fail

```
DELETE FROM orders WHERE ...
```

Pass

```
SELECT ... FROM orders WHERE ...
```

Pattern

Each invariant targets a **specific, cheap-to-check** property. Together they form your first line of defense.

Outline

1. The Eval Mindset
2. Key Definitions
3. Running Example: Support Triage
4. Step 1: Define "Good"
5. Step 2: Build a Golden Dataset
6. Analyze: From Traces to Failure Modes
7. Step 3: Evaluators
8. Step 4: Measure
9. Step 5: Failure Analysis

Step 4: Measure

Measurement means estimating how **prevalent** your failure modes are.

What you produce in this step:

- A small set of metrics (overall + by scenario / failure mode)
- A list of representative failures per failure mode

Unit 1

Always compare against a baseline—even a majority-class prediction gives a sanity check for whether your evaluator measures signal.

Questions to ask while looking at results:

1. Which failure mode is showing up *most*?
2. Which failure mode is most *severe* (red lines)?
3. What examples explain the metric?

Interpretation Pattern

Overall metrics can look fine while high-risk failures are still happening. **Always** report at least one "high risk" scenario group separately.

Outline

1. The Eval Mindset
2. Key Definitions
3. Running Example: Support Triage
4. Step 1: Define "Good"
5. Step 2: Build a Golden Dataset
6. Analyze: From Traces to Failure Modes
7. Step 3: Evaluators
8. Step 4: Measure
9. Step 5: Failure Analysis

Step 5: Failure Analysis (Workflow)

Do this **every time** before changing the system:

1. Pull 5–10 failures from the most frequent or most severe failure mode
2. Write the failure mode in plain English
3. Decide what change targets the root cause:
   - **Add examples** (coverage gap)
   - **Add an invariant** (red line)
   - **Change prompt/tooling** (system behavior)

Principle

Failure analysis is the bridge between **Measure** and **Improve**. Without it, changes are guesses. With it, changes are targeted.

Failure Analysis Example: Data Questions

Failure mode: "missing constraint"

Trace:

- User asked "last 30 days"
- Generated query used all time (no time filter)

Fix (targeting root cause):

1. **Add 5 examples** where the time window is easy to miss
2. **Add an invariant** that checks the query includes a time filter when one is mentioned in the input
3. **Update prompt** to explicitly state: "always include time filters"

The Pattern

One failure mode → one trace → three concrete actions. This is the Analyze–Measure–Improve loop in miniature.

Outline

1. The Eval Mindset
2. Key Definitions
3. Running Example: Support Triage
4. Step 1: Define "Good"
5. Step 2: Build a Golden Dataset
6. Analyze: From Traces to Failure Modes
7. Step 3: Evaluators
8. Step 4: Measure
9. Step 5: Failure Analysis

Agreement and Calibration (Human Labels)

When humans label subjective outputs, **measure agreement**:

Metrics:

- **Agreement rate** (accuracy)
- **Cohen's kappa** (adjusts for chance agreement)

Cohen's kappa:

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

where p_o = observed agreement, p_e = chance agreement.

If agreement is low:

1. Tighten rubric wording
2. Add tie-breaker rules
3. Add borderline examples and how to label them

Warning

Low inter-annotator agreement means your rubric is ambiguous. Fix the rubric before trusting any evaluation results.

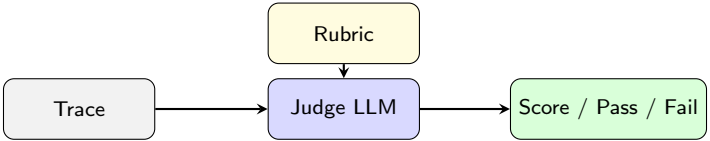
Outline

1. The Eval Mindset
2. Key Definitions
3. Running Example: Support Triage
4. Step 1: Define "Good"
5. Step 2: Build a Golden Dataset
6. Analyze: From Traces to Failure Modes
7. Step 3: Evaluators
8. Step 4: Measure
9. Step 5: Failure Analysis

LLM-as-Judge: Overview

An **LLM judge** is a model that grades another model's outputs using a rubric.

Useful when the output is **subjective** (support replies, summaries, plans) and deterministic checks cannot capture quality.



Important

An LLM judge is **not** "ground truth." It is an evaluator you must **validate and align**.

Building an LLM Judge

What you must build to use an LLM judge:

1. **Rubric** (3–5 criteria): what "good" means for this task
2. **Output schema**: structured output, e.g.:
   - `score` (e.g. 1–5)
   - `passed` (boolean threshold)
   - `rationale` (1–3 sentences)
3. **Judge prompt**: instructions that force the judge to apply the rubric and return only valid structured output
4. **Calibration set**: a small set of labeled examples (human pass/fail or human scores) for agreement checking

Best Practice

Build the calibration set *first*. Without it, you have no way to know whether your judge is doing what you intend.

Aligning a Judge

Alignment means: the judge's decisions match the intent of your rubric and are consistent with human labels on a calibration set.

Check:

- Overall agreement (accuracy; kappa if you have enough labels)
- Disagreement review: read cases where judge $\neq$ human and categorize *why*

Common causes of judge disagreement:

1. Rubric is ambiguous (humans also disagree)
2. Judge is too lenient/strict (threshold mismatch)
3. Judge overweights a criterion (e.g. tone) and ignores another (e.g. safety)

Levers to Improve Alignment (change one at a time)

- Tighten rubric language and add tie-breakers
- Adjust pass threshold
- Add a few labeled exemplars into the judge prompt

Outline

1. The Eval Mindset
2. Key Definitions
3. Running Example: Support Triage
4. Step 1: Define "Good"
5. Step 2: Build a Golden Dataset
6. Analyze: From Traces to Failure Modes
7. Step 3: Evaluators
8. Step 4: Measure
9. Step 5: Failure Analysis

Pitfalls in Evals

Common Pitfalls

- **Goodharting**: optimizing the metric instead of behavior. Recall Goodhart's Law (Unit 1)—slicing your eval by scenario matters at least as much as the headline number.
- **Overfitting**: improvements that only help the golden dataset
- **Metric laundering**: a single number hides real failures
- **Generic metrics**: using measures that aren't relevant to your use case

Countermeasures

- Maintain diverse scenarios (including edge cases)
- Keep some holdout examples for periodic checks
- Keep failure analysis example-driven
- Ensure that your rubric is customized to your applications

The Meta-Principle

Evals are tools, not goals. A passing eval suite means nothing if the underlying behavior is not what your users need.

Outline

1. The Eval Mindset
2. Key Definitions
3. Running Example: Support Triage
4. Step 1: Define "Good"
5. Step 2: Build a Golden Dataset
6. Analyze: From Traces to Failure Modes
7. Step 3: Evaluators
8. Step 4: Measure
9. Step 5: Failure Analysis

Key Takeaways

1. **Evaluation is a method, not a metric**
   - Specify behavior, detect regressions, decide what to improve
2. **Analyze–Measure–Improve is the core loop**
   - Surface failure modes, estimate prevalence, make targeted fixes
3. **Start with deterministic evaluators**
   - Invariants are cheap, fast, and catch regressions
4. **Golden datasets encode your expected behavior**
   - Cover typical, high-risk, ambiguous, and messy cases
5. **Failure analysis before every change**
   - Read failures, categorize root causes, then act
6. **LLM-as-judge requires alignment**
   - Validate against human labels; it is an evaluator, not ground truth
7. **Watch for pitfalls**
   - Goodharting, overfitting, metric laundering, generic metrics

Key Concepts Introduced

- **Analyze–Measure–Improve** — the core eval lifecycle
- **Trace** — full record needed to evaluate one case
- **Failure mode** — recurring way the system fails
- **Failure-mode taxonomy** — stable set after merging open codes
- **Evaluator** — per-example check producing metrics/pass-fail
- **Invariant** — deterministic evaluator that should always hold
- **Red lines** — unacceptable failures
- **LLM-as-judge** — model that grades outputs using a rubric
- **Calibration set** — labeled examples for judge agreement checking